

Part C Project Progress Report

M Ahsan Al Mahir

Project Overview

Most modern real-time operating systems (RTOS) for small microcontrollers allow a limited number of processes to run simultaneously, typically defined at compile time. However, this restricts the flexibility to create processes dynamically. In contrast, one of the earliest examples of an RTOS, deployed on the Apollo Guidance Computer (AGC), allowed latency-insensitive jobs to be declared at runtime while processes with stricter timing constraints were defined at compile time. The queue of processes scheduled to run at predefined intervals was called the `WAITLIST`. Longer `EXECUTIVE` jobs could be created dynamically with varying priorities, often initiated by `WAITLIST` jobs, and were executed using the remaining available processor time.

This flexible design made the AGC highly fault-tolerant. Famously, during the Apollo 11 landing, the AGC in the lunar module ran out of memory for new `EXECUTIVE` jobs, triggering a 1202 alarm. While low-priority tasks, such as updating the astronauts' displays, failed to execute, higher-priority jobs that controlled the spacecraft continued unaffected, allowing the crew to land safely.

The goal of this project is to explore how this scheduling algorithm can be used to implement an operating system on a small microcontroller that is rich in features that are normally found in more robust operating systems designed for actual computers.

Progress

Currently I have a working implementation of the operating system with the following list of features:

Scheduler

The scheduler is implemented following the same algorithm as in the AGC. The core scheduler is priority based, where the highest priority process has sole control over the CPU. When a higher priority process is scheduled, it has to wait until the currently running process gives up execution. This avoids some race conditions in preemptive scheduling by allowing the processes to define critical sections.

To allow processes to schedule deadline-critical tasks, there is a `waitlist` queue of tasks that are run preemptively at certain intervals. These tasks are run no matter what the currently running process is, ensuring the completeness of time-critical tasks.

Processes utilize this `waitlist` queue to implement sleeping/wake up mechanisms. When a process wants to pause its execution, it goes into sleep by negating its priority, and then it schedules a waitlist task to act as an alarm and wake the process up at a certain interval in the future.

This sleep/wake up mechanism is also used to implement a bare-bone version of `mutex`, which internally stores a FIFO queue of processes waiting on the `mutex`.

Memory management

A simple freelist-based memory allocator implementation is used to allocate run-time memory like process stack, dynamically allocated objects, memory mapping from files, etc. Dynamic memory allocated by processes is kept track of inside the process descriptors, which are then cleared at process termination to reduce the risk of memory leakage. Process stacks are allocated by a separate instance of the same allocator implementation, which allocates stacks from the rear end of the RAM, while normal dynamic memory allocations are allocated from the front end.

Drivers

Currently there are a few driver implementations available with the operating system:

- **UART:** The serial driver allows communication on the UART bus. This driver is heavily used to print debug information on the minicom window, and also to implement a `shell`-like program. More on `shell` is below.
- **Timer:** The timer drivers increment the program clock and run the waitlist tasks at a set interval. This driver also periodically runs some sanity checks for the processes to detect stack overflows.
- **Display:** The display driver gives an easy-to-use interface for the processes to display images on the 5×5 LED display. The display process runs at low priority to update the display periodically.
- **I2C:** There are two separate implementations for communicating with the I2C devices:
 - The `i2c::sync` namespace provides a synchronized, busy-wait implementation, which is used by the kernel to ensure proper setup during initialization.
 - The `i2c::async` namespace provides an asynchronous implementation for general I2C communication. This implementation uses the I2C interrupts to asynchronously handle the communication, while letting other processes proceed while the current process sleeps and waits for the communication to complete.
- **Accelerometer:** The driver for the accelerometer communicates through the I2C master. It can be used by any process to receive the data collected by the accelerometer.

File System

A block device file system is implemented on the flash memory. This file system allows persistent storage for files of size a multiple of 1KB. However, due to the limited lifespan of flash memory, and the incredibly inefficient erase/write sequences required to perform writes to the flash memory, this file system is not ideal for files that are written to more often than they are read from.

To counter this, an FRAM device was added to the micro:bit device, which allows for non-volatile random access to about 32KB of memory. This FRAM operates through I2C and is quite

efficiently read from and written to. On this FRAM, a more efficient file system is implemented, which is used throughout the rest of the operating system for more consistent storage.

Due to the similarities between these two implementations, several design decisions were made to make the implementation more maintainable. The file system implementation is completely header-only, which uses C++ templates to create an abstraction of a block-device file system. This implementation is then instantiated with specific methods for communication with the flash and FRAM memory.

Recovery Mechanism

I have recreated a modified version of the AGC's recovery mechanism, which allows a simple interface to protect a critical section of code against unexpected resets. There are a few levels to this:

- A process can protect itself, or a part of itself against unexpected resets. On a reset, a protected process is re-scheduled, with some parameters passed to the newly scheduled process that communicates what the state of the previous run was, so that the newly scheduled process can decide where to start the recovery from.
- A function can protect a part of itself by registering a recovery function, which is run when the system recovers, and recovers the state of the previously running function. This mechanism gives reset protection for critical functions that must not be disturbed unexpectedly.

The recovery mechanism requires persistent storage to recover from a power off. For this, the recovery mechanism stores its information on a file on the RAM.

Signals

An implementation of signals is also present in this operating system. Signals operate mostly like software-generated interrupts. Using this, a process can send commands to other processes. Signals awaiting to be handled by a process are handled when the scheduler switches into that process the next time.

Shell

A modified version of the Unix Shell is added to the operating system. On the minicom window, this looks just like a normal shell, albeit with much reduced capacity. Commands can be written to the shell, and when enter is pressed, the corresponding process is scheduled with a predefined priority. The process receives the arguments passed to the command. Currently there are a few shell commands that are noteworthy:

- `echo`: just echoes the argument.
- `trace`: dumps some debug information about the currently running processes, the files that are open, and the memory locations in use.
- `accel`: a mini (useless) compass is implemented using the readings from the accelerometer to show an arrow on the display that always points towards the ground.

- `pkill`: a command that sends a `SIGTERM` signal to the target process, effectively terminating it earlier than its normal exit.
- `calc`: a mini calculator.
- `heart`: displays a beating heart on the LEDs.
- `show`: displays an English sentence on the LEDs.

Feedback

Over the winter vacation and the Hilary term, there has been a lot of progress on this project. I am quite satisfied with the progress so far. I have met with my supervisor, Prof. Alex Rogers, 4 times this term so far. His guidance and feedback has been very helpful throughout the course of the project.